

Enterprise Operations Assistant

Implementation Guide and Solution Approach

A practical guide for designing, implementing, testing, and demonstrating the capstone project

Purpose of this guide: This document converts the capstone requirements into an implementation-oriented plan. It explains what must be built, how the components can be organized, what state should flow through the LangGraph workflow, how the human-approval checkpoint can be handled, and how the implementation can be tested and demonstrated. It is a guide rather than a single prescribed implementation; equivalent designs are acceptable where they satisfy the original requirements.

1. Project Overview

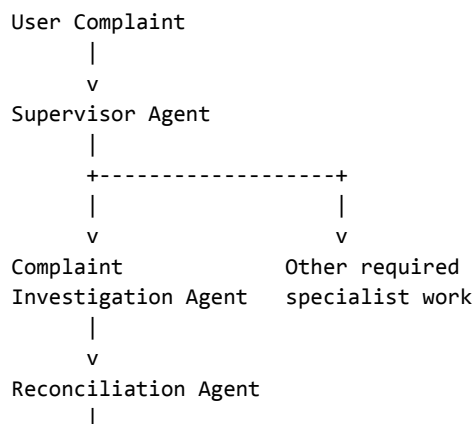
The project is an Enterprise Operations Assistant that receives customer complaints in natural language, investigates them using internal and external data, reconciles discrepancies, recommends a resolution, and records the outcome. Financial adjustments must stop for explicit supervisor approval before the financial operation is executed.

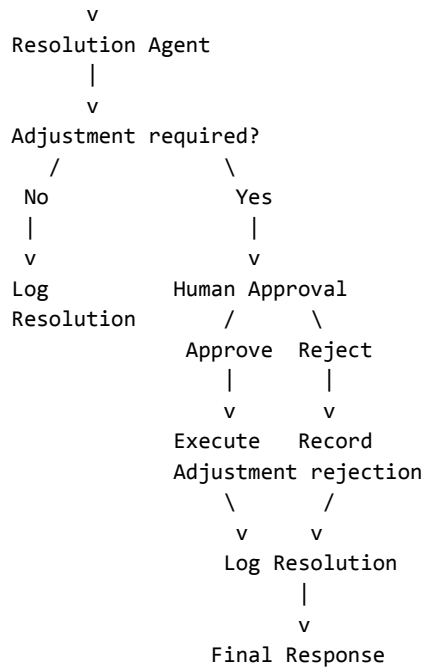
The reference scenario is a complaint from customer C1024: the internal billing system shows ₹12,500 while the external billing system reports ₹10,000. The expected investigation therefore identifies the customer and invoice, retrieves both billing records, detects a ₹2,500 difference, recommends an adjustment, pauses for approval, and resumes according to the approval decision.

Source basis: The original capstone defines the business scenario, required agents, tools, human-in-the-loop behavior, observability, evaluation scenarios, technical expectations, and deliverables. This guide preserves those requirements and adds an implementation approach.

2. What the Finished System Should Look Like

A useful target architecture is:





The exact graph can differ. The important requirement is meaningful multi-agent coordination and state-based workflow management. The supplied project diagram on page 5 shows the same broad structure, including SQLite, a mock billing API, policy/guideline sources, supervisor coordination, conditional routing, human approval, logging, and the final response.

3. Requirement-to-Implementation Map

Requirement	Suggested implementation
Natural-language complaint	Initial graph input containing the complaint text
Complaint analysis	Complaint Investigation Agent using structured tools
Customer/billing database	SQLite with Customers, Invoices, and Adjustments tables
Third-party billing API	Small mock REST service exposing GET /billing/{invoice_id}
At least three specialized agents	Investigation, Reconciliation, and Resolution agents
Supervisor	LangGraph supervisor node/agent that selects or routes work
Tools	LangChain tools with typed/structured input and output
Stateful workflow	Typed shared state passed between graph nodes
Conditional routing	Graph routing based on reconciliation/resolution state
Human approval	Interrupt/pause checkpoint before financial execution
Secure/auditable logging	SQLite resolution log capturing investigation, decision, approval, and status
RAG/retrieval where appropriate	Retrieve billing policies, adjustment rules, and escalation criteria
Observability	LangSmith or Langfuse tracing
Automated evaluation	Five or more repeatable scenarios with accuracy and latency metrics

4. Recommended Implementation Strategy

Do not attempt to build the complete system in one step. Implement it incrementally.

1. Build and populate the SQLite database.
2. Build and test the mock billing REST API independently.
3. Implement the business tools and verify that each tool returns structured data.
4. Define the shared LangGraph state.
5. Implement the Complaint Investigation Agent.
6. Implement the Reconciliation Agent.
7. Implement the Resolution Agent.
8. Add the Supervisor and conditional routing.
9. Add the human-approval checkpoint before financial adjustment execution.
10. Add resolution logging.
11. Add policy retrieval/RAG if used by the resolution logic.
12. Instrument the complete workflow with LangSmith or Langfuse.
13. Create the five required evaluation scenarios.
14. Automate the evaluation and generate the results.
15. Prepare the README and demonstration.

Recommended development rule: Test every layer independently before integrating it into the graph. For example, verify `get_invoice()` and `GET /billing/{invoice_id}` before asking an agent to call them.

5. Step 1 — Design the SQLite Database

Start with the operational data model. The original requirement suggests Customers, Invoices, and Adjustments.

Table	Important fields	Purpose
Customers	customer_id, name, email, account_status	Customer identity and account information
Invoices	invoice_id, customer_id, invoice_amount, invoice_status, invoice_date	Internal invoice information
Adjustments	adjustment_id, customer_id, invoice_id, amount, status, reason	Financial adjustment lifecycle
ResolutionLogs	customer_id, complaint, invoice_id, investigation_result, reconciliation_result, recommended_resolution, adjustment_amount, approval_decision, final_status, timestamp	Audit trail for completed complaints

A minimal starting schema could be:

```
CREATE TABLE customers (
  customer_id TEXT PRIMARY KEY,
  name TEXT,
  email TEXT,
  account_status TEXT
);

CREATE TABLE invoices (
  invoice_id TEXT PRIMARY KEY,
  customer_id TEXT,
```

```

        invoice_amount REAL,
        invoice_status TEXT,
        invoice_date TEXT,
        FOREIGN KEY(customer_id) REFERENCES customers(customer_id)
    );

    CREATE TABLE adjustments (
        adjustment_id TEXT PRIMARY KEY,
        customer_id TEXT,
        invoice_id TEXT,
        amount REAL,
        status TEXT,
        reason TEXT
    );

```

The ResolutionLogs table can be added once the core workflow is working.

6. Step 2 — Create Representative Sample Data

Do not create only one happy-path record. Build data that supports the evaluation scenarios.

Scenario	Internal data design	External/API design	Expected outcome
Billing mismatch	Invoice ₹12,500	Billing API ₹10,000	₹2,500 discrepancy; approval required if adjustment is recommended
Duplicate charge	Two relevant charges/records	External billing confirms duplicate or corresponding transaction state	Identify duplicate and recommend resolution
Correct billing	Invoice ₹10,000	API ₹10,000	No adjustment required
Financial adjustment	Data that produces an adjustment recommendation	Verified external amount differs	Workflow pauses for human approval
Invalid customer/invoice	Missing customer or invoice	N/A or no valid external lookup	Graceful error handling

The exact sample values beyond the reference C1024/INV10045 example are implementation choices. The important point is that each evaluation scenario must produce a meaningful, testable workflow outcome.

7. Step 3 — Build the Mock Billing API

Implement a small REST service representing the external billing system. A lightweight FastAPI application is one possible approach.

```
GET /billing/{invoice_id}
```

Example:

```
GET /billing/INV10045
```

Response:

```

{
  "invoice_id": "INV10045",
  "customer_id": "C1024",
  "amount": 10000,
  "currency": "INR",
  "status": "PAID",

```

```
"billing_date": "2026-09-20"
}
```

Keep the API deliberately small. Its purpose is to demonstrate REST integration and create a second source of truth that the assistant must compare with SQLite.

Important: The external API should be treated as a separate system. Do not simply call the SQLite function from the API tool; the point is to demonstrate an actual external-service boundary.

8. Step 4 — Expose Enterprise Operations as Tools

The capstone explicitly requires appropriate enterprise operations to be exposed as tools. Implement each operation as a small, deterministic function and expose it through LangChain's tool mechanism.

```
get_customer(customer_id)
get_invoice(invoice_id)
get_customer_invoices(customer_id)
get_external_billing(invoice_id)
reconcile_invoice(invoice_id)
create_financial_adjustment(customer_id, invoice_id, amount, reason)
update_adjustment_status(adjustment_id, status)
log_resolution(customer_id, complaint, resolution)
```

A useful tool-design rule is:

- One tool should perform one business operation.
- Inputs should be explicit and validated.
- Return structured dictionaries/models rather than long prose.
- Do not hide business decisions inside generic tools.
- Financial-write tools should be called only after the approval state is satisfied.

For example, a tool result should look more like this:

```
{
  "invoice_id": "INV10045",
  "customer_id": "C1024",
  "amount": 12500,
  "status": "PAID"
}
```

rather than returning a paragraph that another LLM must parse.

9. Step 5 — Define the Shared LangGraph State

Before building the agents, define the state that travels through the graph. This is one of the most important design decisions because the agents should communicate primarily through structured state.

```
class OperationsState(TypedDict, total=False):
    complaint: str

    customer_id: str
    invoice_id: str
```

```
customer: dict
internal_invoice: dict
external_invoice: dict

investigation_result: dict
reconciliation_result: dict
resolution_result: dict

discrepancy_detected: bool
adjustment_required: bool
adjustment_amount: float

approval_required: bool
approval_status: str

adjustment_result: dict
final_status: str
error: str
```

The exact state shape is your design choice. Keep it explicit and avoid putting large unstructured conversation transcripts into every field.

10. Step 6 — Implement the Complaint Investigation Agent

The Investigation Agent should answer: What is the complaint, who is involved, what invoice is relevant, and what information is needed?

16. Read the natural-language complaint.
17. Identify the customer ID if supplied.
18. Find the relevant invoice or customer invoices.
19. Retrieve the internal invoice.
20. Call the external billing tool when the complaint requires comparison.
21. Write structured investigation results into graph state.

A possible result:

```
{
  "customer_id": "C1024",
  "invoice_id": "INV10045",
  "internal_amount": 12500,
  "external_amount": 10000,
  "investigation_complete": True
}
```

Design choice: The agent may use an LLM to interpret the complaint and select tools, but deterministic lookups should remain deterministic. Do not ask the LLM to invent database values.

11. Step 7 — Implement the Reconciliation Agent

The Reconciliation Agent should compare the internal and external records and produce a structured result.

```
{
  "internal_amount": 12500,
  "external_amount": 10000,
  "difference": 2500,
  "discrepancy_detected": True,
  "status": "DISCREPANCY_DETECTED"
}
```

The arithmetic and comparison should ideally be deterministic code rather than an LLM calculation. The LLM can explain or interpret the result, but the system should not depend on probabilistic arithmetic for a financial amount.

12. Step 8 — Implement the Resolution Agent

The Resolution Agent reviews the investigation and reconciliation results and produces a recommendation.

```
{
  "recommended_action": "CREATE_FINANCIAL_ADJUSTMENT",
  "adjustment_required": True,
  "adjustment_amount": 2500,
  "reason": "Internal invoice amount differs from verified third-party billing amount."
}
```

If policy information is available through retrieval, the Resolution Agent can consult the relevant billing/adjustment policy before making its recommendation.

13. Optional but Useful — Policy Retrieval / RAG

The technical expectations mention RAG or retrieval-based access where appropriate. A sensible use is a small policy knowledge base containing billing policies, adjustment rules, and escalation criteria.

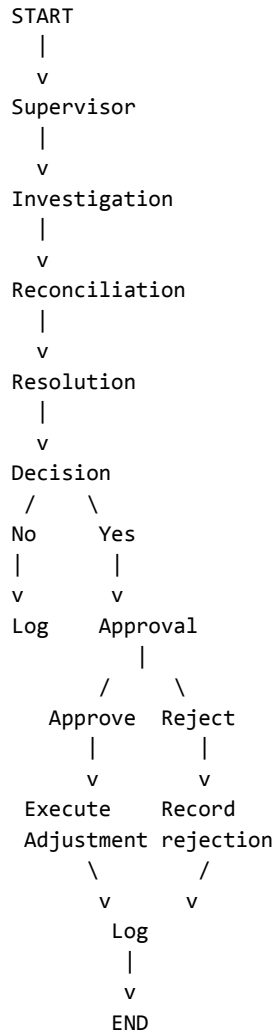
```
Question:
"What is the permitted resolution for a verified billing discrepancy?"

|
v
Policy Retriever
|
v
Relevant policy passages
|
v
Resolution Agent
|
v
Recommendation
```

Keep the retrieval scope focused. The capstone does not prescribe a particular vector database or embedding implementation, so use a simple retrieval approach that can be clearly demonstrated.

14. Step 9 — Build the Supervisor Workflow

Once the three specialist agents work independently, place them inside a LangGraph workflow. The supervisor should control the sequence based on state rather than forcing every complaint through an identical path.



The supervisor may route back for additional information if the current state is insufficient. That demonstrates the dynamic coordination expected by the project.

15. Step 10 — Human-in-the-Loop Design

The approval boundary must be explicit. The critical rule is: a recommendation is not an execution.

Stage	Allowed?	Reason
Investigate customer	Yes	Read-only operation
Retrieve invoice	Yes	Read-only operation
Call external billing API	Yes	Read-only external verification
Calculate discrepancy	Yes	Analysis
Recommend adjustment	Yes	Recommendation only
Create financial adjustment before approval	No	Violates the human-approval requirement

Create financial adjustment after approval	Yes	Approval requirement satisfied
Log final result	Yes	Required audit operation

A practical LangGraph design is to place an interrupt/pause immediately before the financial-write node. The paused state should contain the customer, invoice, internal amount, external amount, difference, finding, recommendation, and reason required by the project.

Demonstration tip: During the demonstration, visibly show that the workflow stops before `create_financial_adjustment()`. Then approve the action and show the graph resuming and executing the tool.

16. Step 11 – Approval Outcomes

Handle both outcomes explicitly.

Decision	Next step	Final status
APPROVE	Execute adjustment, update status, log resolution	Adjustment processed / completed
REJECT	Do not execute adjustment; record rejection and reason	Adjustment rejected / no adjustment executed

The rejected path is just as important as the approved path because it demonstrates that the human decision actually controls execution.

17. Step 12 – Error Handling

The invalid customer/invoice evaluation scenario requires graceful error handling.

- Validate required IDs before making downstream calls.
- Return structured tool errors rather than raw stack traces to the agent.
- Do not call the external API if the invoice cannot be established.
- Store an error status in graph state.
- Produce a useful final response explaining what could not be verified.
- Log the failed workflow where appropriate.

```
{
  "final_status": "INVESTIGATION_FAILED",
  "error": "Invoice INV99999 was not found for customer C1024"
}
```

18. Step 13 – Secure/Auditable Resolution Logging

The resolution log should be sufficient to reconstruct what happened without relying solely on LLM traces.

Field	Example
Customer ID	C1024
Complaint	Customer reports incorrect invoice amount
Invoice ID	INV10045
Investigation result	Internal and external records retrieved
Reconciliation result	₹2,500 discrepancy detected

Recommended resolution	Create ₹2,500 adjustment
Adjustment amount	2500
Approval decision	APPROVED
Final status	ADJUSTMENT COMPLETED
Timestamp	2026-09-24T...

19. Step 14 – Observability with LangSmith or Langfuse

Instrument the application so a trace can follow the complete request. The project asks for coverage of the user request, supervisor decisions, agent execution, LLM calls, tool calls, tool inputs/outputs, state transitions, human approval, and the final response.

```
User request
-> Supervisor decision
-> Investigation agent
-> get_customer
-> get_invoice
-> get_external_billing
-> Reconciliation agent
-> Resolution agent
-> Approval checkpoint
-> create_financial_adjustment
-> log_resolution
-> Final response
```

Use the traces to diagnose:

- Incorrect agent selection
- Incorrect tool invocation
- Invalid tool parameters
- Unexpected state transitions
- Excessive latency
- Incorrect final responses

20. Step 15 – Automated Evaluation

Build the evaluation harness after the basic workflow works. Each scenario should be repeatable.

Scenario	Input	Important assertions
Billing mismatch	C1024 complaint	Correct invoice; external verification; discrepancy detected; correct difference
Duplicate charge	Duplicate-billing complaint	Duplicate identified; appropriate recommendation
Correct billing	Complaint with matching amounts	No discrepancy; no financial adjustment
Financial adjustment	Complaint that requires adjustment	Workflow pauses; approval required; execution only after approval
Invalid customer/invoice	Unknown ID	Graceful error; no invalid downstream financial action

Recommended evaluation structure:

```

for scenario in scenarios:
    start_time = time.perf_counter()

    result = run_workflow(
        scenario.complaint,
        scenario.approval_decision
    )

    latency = time.perf_counter() - start_time

    assert_expected_behavior(result, scenario.expected)
    record_result(scenario.name, result, latency)

```

The capstone specifically asks for accuracy and latency measurements. Do not hard-code the example results shown in the original document; generate results from the implementation.

21. Suggested Project Structure

```

enterprise_operations_assistant/
|
+-- app/
|   +-- main.py
|   +-- config.py
|   |
|   +-- agents/
|       +-- investigation.py
|       +-- reconciliation.py
|       +-- resolution.py
|       +-- supervisor.py
|       |
|   +-- graph/
|       +-- state.py
|       +-- workflow.py
|       +-- routing.py
|       |
|   +-- tools/
|       +-- customer_tools.py
|       +-- invoice_tools.py
|       +-- billing_tools.py
|       +-- adjustment_tools.py
|       +-- logging_tools.py
|       |
|   +-- db/
|       +-- schema.sql
|       +-- seed.py
|       +-- operations.db
|       |
|   +-- retrieval/
|       +-- policies/
|       +-- retriever.py
|       |
|   +-- models/
|       +-- schemas.py
|
+-- mock_billing_api/
|   +-- main.py
|   +-- data.py
|

```

```

+-- evaluations/
|   +-- scenarios.py
|   +-- evaluate.py
|   +-- results/
|
+-- README.md
+-- requirements.txt
+-- .env.example

```

This structure is only a suggested organization. A different structure is acceptable if the responsibilities remain clear.

22. Recommended Implementation Order

Phase	Build	Verification before moving on
1	SQLite schema and seed data	CRUD queries return expected records
2	Mock billing API	GET /billing/{invoice_id} works
3	LangChain tools	Each tool returns structured data
4	Shared graph state	State can hold all required workflow information
5	Investigation agent	Correct customer/invoice/external lookup
6	Reconciliation agent	Correct discrepancy and amount
7	Resolution agent	Correct recommendation and adjustment flag
8	Supervisor graph	Correct routing for happy path
9	Human approval	No financial write before approval
10	Logging	Completed and rejected cases are auditable
11	Observability	Full trace visible
12	Evaluation	Five scenarios run automatically
13	README/demo	Another person can set up and run the system

23. What Should Be Deterministic vs. LLM-Driven?

A strong implementation does not ask the LLM to do everything.

Responsibility	Preferred approach	Reason
Interpret complaint	LLM/agent	Natural-language understanding
Choose information/tools	Agent + supervisor	Dynamic workflow requirement
Retrieve customer/invoice	Deterministic tool	Database is authoritative
Call billing API	Deterministic tool	External system operation
Calculate monetary difference	Deterministic code	Financial arithmetic should be reliable
Compare structured amounts	Deterministic code	Avoid probabilistic comparison
Interpret business policy	LLM + retrieval	Policy text may require interpretation
Recommend resolution	Resolution agent	Requires contextual reasoning
Decide whether approval is required	Explicit workflow rule plus agent result	Safety/control boundary
Execute financial adjustment	Deterministic tool after approval	Controlled side effect
Audit logging	Deterministic tool	Reliable record keeping

Core principle: Use the LLM for interpretation, planning, and contextual reasoning; use deterministic tools and graph rules for data access, arithmetic, side effects, and control boundaries.

24. Common Mistakes to Avoid

- Building three independent agents without meaningful LangGraph coordination.
- Hard-coding the entire workflow as a fixed linear sequence and calling it dynamic.
- Allowing the LLM to directly modify the database.
- Calling `create_financial_adjustment()` before approval.
- Using an LLM to calculate the ₹2,500 difference when deterministic arithmetic is available.
- Returning unstructured paragraphs from every tool.
- Ignoring the rejected-approval path.
- Treating the external API as if it were just another SQLite query.
- Logging only the final answer and losing the underlying investigation/reconciliation decisions.
- Adding tracing only at the end instead of instrumenting the workflow.
- Creating evaluation scenarios but manually assigning the expected results after execution.
- Hard-coding the evaluation numbers shown in the capstone instead of generating measurements.
- Building a large RAG system that does not materially help the resolution decision.
- Writing a README that describes the architecture but does not contain setup and run instructions.

25. A Good Demonstration Sequence

A concise demonstration can follow the complete lifecycle.

22. Start the SQLite database and mock billing API.
23. Show the sample customer/invoice data.
24. Submit the C1024 billing-discrepancy complaint.
25. Show the supervisor and investigation trace.
26. Show the internal invoice lookup and external API call.
27. Show the reconciliation result: ₹12,500 vs ₹10,000 and ₹2,500 difference.
28. Show the resolution recommendation.
29. Show that the graph pauses before the financial adjustment.
30. Approve the adjustment.
31. Show `create_financial_adjustment()` executing after approval.
32. Show the resolution log.
33. Show the final response.
34. Run the evaluation suite and show the results for all five scenarios.
35. Open the LangSmith/Langfuse trace and show the end-to-end execution.

26. Definition of Done

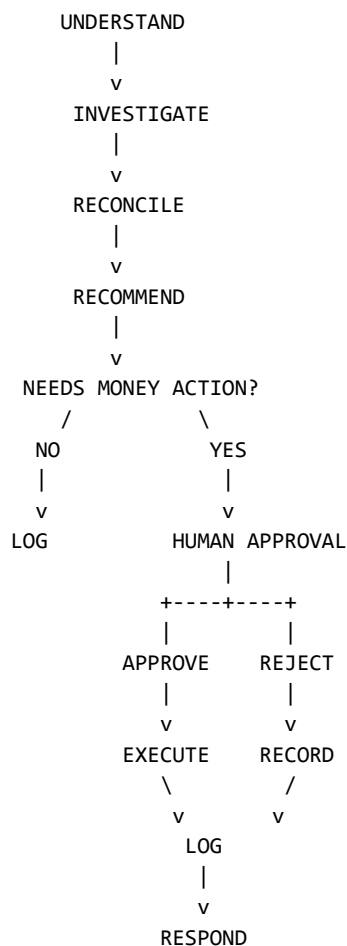
The implementation can be considered complete when all of the following are true:

- A natural-language complaint can start the workflow.
- At least three specialized agents are implemented.
- A supervisor coordinates the agents through LangGraph.
- The graph maintains structured state.
- SQLite provides internal operational data.

- The mock REST API provides external billing information.
- Agents access enterprise operations through tools.
- Internal and external billing data are reconciled.
- A resolution recommendation is generated.
- Financial adjustments pause for explicit human approval.
- Approved adjustments execute only after approval.
- Rejected adjustments do not execute.
- Every completed complaint is logged.
- The workflow is fully observable through LangSmith or Langfuse.
- At least five automated scenarios are evaluated.
- Accuracy and latency are measured from actual execution.
- The README documents architecture, setup, agents, tools, scenarios, responses, and evaluation results.

27. Final Mental Model

When implementing the project, think of the system as an operational decision workflow rather than a collection of chatbots.



The strongest implementation will make each transition explicit: what the current state is, which agent or node is responsible, which tools it can use, what information it adds to state, what condition controls the next transition, and where a human must take control.

28. Deliverables Checklist

#	Deliverable	Status to verify
1	Python source code	Complete and runnable
2	SQLite database with sample data	Included and reproducible
3	Mock third-party billing API	Runnable independently
4	Sample complaint scenarios	At least five representative cases
5	LangGraph workflow	Stateful and conditionally routed
6	Specialized agents and tools	Investigation, reconciliation, resolution, supervisor
7	Human-approval workflow	Approval before financial execution
8	LangSmith/Langfuse traces	End-to-end trace coverage
9	Automated evaluation script	Repeatable
10	Evaluation results	Accuracy and latency included
11	README	Architecture, setup, responsibilities, tools, examples, results

Appendix A — Reference Scenario Walkthrough

For the document's main example:

Step	Input/Action	Result
1	Complaint received	C1024 reports ₹12,500 vs ₹10,000
2	Investigation	Customer and invoice identified
3	Internal lookup	INV10045 = ₹12,500
4	External lookup	INV10045 = ₹10,000
5	Reconciliation	₹2,500 discrepancy
6	Resolution	Recommend ₹2,500 adjustment
7	Control point	Pause for supervisor approval
8	Approval	Supervisor chooses APPROVE or REJECT
9	Financial action	Execute only if approved
10	Logging	Store full operational outcome
11	Response	Return concise investigation/resolution summary

Appendix B — Suggested README Contents

36. Project overview
37. Architecture diagram
38. Technology stack
39. Prerequisites
40. Environment variables
41. Database setup
42. Mock API setup
43. How to run the application
44. Agent responsibilities
45. Tool descriptions
46. LangGraph state and routing
47. Human-approval mechanism
48. Policy/RAG approach, if implemented

- 49. Sample requests and responses
- 50. Observability setup
- 51. Evaluation scenarios
- 52. Evaluation results
- 53. Known limitations

End of Implementation Guide